

Your Neovim, For Real Work

A field guide to Raf's kickstart-based setup — navigation, editing, git, and debugging

July 2026

How this guide works

This is written against **your actual config** (`~/Developer/dotfiles/nvim`, `kickstart.nvim` base). Every keybinding here exists in your setup today. It's organized around *dev actions* — find, read, edit, fix, refactor, commit, debug — not around plugins.

Three things to internalize before anything else:

1. **Leader is Space.** Almost everything you've customized hangs off it.
2. **which-key is your built-in tutor.** Press `Space` and *wait* — a menu of every chord appears instantly (you set `delay = 0`). When you forget a binding, don't reach for this PDF: press `Space` and read.
3. **Space s k fuzzy-searches every keymap** in your session, with descriptions. This is the single best discovery tool you have.

Your setup's big idea: **one VSCode-style left sidebar** (`Snacks.nvim`) that swaps between file explorer, file finder, grep, buffers, and git status — instead of a pile of floating windows. You built this deliberately; lean on it.

1. Finding anything (the sidebar pickers)

All of these open in (or swap into) the single left sidebar. If the picker is already open, the binding just focuses it. `q` closes the sidebar; `Esc` from the input jumps you back to normal mode inside it. The sidebar **stays open** after you pick a file — that's your config, not an

accident.

Key	What it finds
Space e	File explorer (tree)
Space E	File explorer rooted at cwd
Space s f or Space F	Files by name (includes hidden files)
Space s g or Space G	Live grep across the project
Space s w	Grep the word under the cursor
Space Space	Open buffers
Space b e	Buffer explorer (sidebar)
Space /	Fuzzy search lines in the current buffer
Space s /	Grep only the open files
Space s .	Recent files
Space s r	Resume the last picker where you left it
Space s d	All diagnostics, project-wide
Space s k	Keymaps (your discovery tool)
Space s h	Neovim help tags
Space s n	Your Neovim config files
Space g e	Git status (changed files) in the sidebar

Power move — picker to quickfix: inside a Snacks picker, `Ctrl-q` dumps the results into the quickfix list. Grep for something, `Ctrl-q`, then walk matches with `:cnext` / `:cprev`, or mass-edit with `:cfd %s/old/new/ge | update`. This turns “search” into “operate on every result.”

Note on grep flavors: your sidebar grep (Space s g) treats the pattern as a regex (it’s ripgrep underneath). Telescope is still installed and its `:Telescope live_grep` is configured with `--fixed-strings` — literal matching, no regex escaping needed. Two tools, pick per job.

Habit to build

Stop scrolling the file tree to open files. The tree (Space e) is for *orienting* in an unfamiliar area; Space F (name) and Space G (content) are for *going places*. If you know anything about the target — its name, a string inside it — the pickers are always faster.

2. Reading and navigating code (LSP)

Your LSP servers (auto-installed via Mason): **gopls** (with staticcheck, gofumpt, inlay hints), **ts_ls** + **eslint**, **rust_analyzer**, **pyright**, **clangd**, **bashls**, **lua_ls**. Results lists open in Telescope.

Key	Action
<code>g r d</code>	Go to definition
<code>g r r</code>	List all references
<code>g r i</code>	Go to implementation
<code>g r t</code>	Go to type definition
<code>g r D</code>	Go to declaration (C/C++ header)
<code>g r n</code>	Rename symbol across the project
<code>Space c a</code>	Code action (auto-import, fix, refactor)
<code>K</code>	Hover docs for symbol under cursor (built-in)
<code>g 0</code>	Fuzzy-find symbols in this file
<code>g W</code>	Fuzzy-find symbols in the workspace
<code>Ctrl-t / Ctrl-o</code>	Jump back after a goto
<code>Ctrl-i</code>	Jump forward again
<code>Space t h</code>	Toggle inlay hints (great for Go)

The core loop to drill

`g r d` in, read, `Ctrl-t` back. Then `g r r` to see every caller, pick one from the Telescope list, `Ctrl-t` back again. Navigating by *meaning* (definitions, references, symbols) instead of by *files* is the single biggest speed upgrade over a VSCode-style workflow. `g 0` deserves special attention: in a 800-line file, it's how you jump straight to the function you want.

In any Telescope list, `Ctrl-Enter` opens the selection in a **vertical split** (your custom mapping) — perfect for “definition on the right, call site on the left.”

Folds as a skimming tool (nvim-ufo)

Treesitter-powered folding is on with the fold column visible.

Key	Action
z M	Close all folds — see the file’s skeleton
z R	Open all folds
z a	Toggle fold under cursor (built-in)
z p	Peek inside a fold without opening it — or LSP hover if not on a fold

Open an unfamiliar file, hit z M, and you get a table of contents made of function signatures. Drill into one with z a or peek with z p.

3. Editing faster (where vim skill actually lives)

Text objects — mini.ai

You have enhanced a/i text objects. The grammar is `operator + a/i + object`:

Type this	You get
c i "	Change inside the quotes
d a)	Delete around the parentheses (including them)
v i b	Select inside the nearest bracket pair
y a f	Yank around the function call under cursor
c i a	Change the current argument in a call

mini.ai adds n (next) and l (last/previous): c i n q = change inside the *next* quote, even if your cursor isn’t in it yet. f (function call) and a (argument) are the two dev-specific objects worth drilling — c i a to rewrite one argument in a long call is something people do fifty times a day the slow way.

Surround — mini.surround

Type this	Effect
<code>s a i w "</code>	Surround the word with quotes
<code>s a i w)</code>	Wrap word in parens: <code>x</code> becomes <code>(x)</code>
<code>s d "</code>	Delete surrounding quotes
<code>s r " '</code>	Replace <code>"..."</code> with <code>'...'</code>

In visual mode, select something then `s a )` wraps the selection. Changing `'` to ``` for a template literal is `s r ' “` — no cursor gymnastics.

Your quality-of-life mappings

Key	Action
<code>Cmd-/ Cmd-S</code>	Toggle comment (normal, visual, and insert mode) Save (works in insert mode too)
<code>p</code> (visual)	Paste over a selection <i>without</i> clobbering your yank — you can paste the same thing repeatedly
<code>g c c / g c</code>	Comment line / selection (the underlying vim way)
<code>Space r p</code>	Copy the file's relative path to clipboard
<code>Space r P</code>	Copy the absolute path
<code>Esc</code>	Clear search highlighting

Also live in your config: **autopairs** (brackets close themselves), **guess-indent** (matches each file's existing indentation), **highlight-on-yank** (flash shows exactly what you copied), and **:s live preview** (`inccommand=split` — a substitute command shows its results in a split as you type it).

Completion (blink.cmp)

Key	Action
<code>Enter</code>	Accept completion (auto-imports if the LSP supports it)

Key	Action
Tab / Shift-Tab	Next / previous item
Ctrl-Space	Open menu, or show docs for the selected item
Ctrl-e	Dismiss the menu
Ctrl-k	Toggle signature help

Signature help also pops automatically as you type call arguments.

4. Fixing what's broken (diagnostics, format, lint)

Diagnostics

Key	Action
g r e	Next diagnostic (your custom map)
g r E	Previous diagnostic
l d / l D	Next / previous diagnostic (built-in, same thing)
Ctrl-w d	Show the diagnostic under cursor in a float (built-in)
Space q	All buffer diagnostics into the location list
Space s d	Project-wide diagnostics in the picker
Space c a	Code action — often the actual fix

Flow: **g r e** to the squiggle, **Space c a** to see what the LSP offers, pick a fix, repeat. For a red-riddled file after a refactor, **Space s d** gives the full damage report in one list.

Formatting (conform.nvim) — know your own rules

Format-on-save is on (500ms timeout), and **Space f** formats on demand. But your per-language config has behavior worth knowing:

- **Go** → goimports then gofmt. Imports fix themselves on save.
- **Lua** → stylua.
- **C/C++** → clang-format.

- **JS/TS/JSX/TSX** → **eslint_d** only, and *only if an ESLint config exists* above the file (monorepo-aware — each file resolves to its nearest sub-project config). If a project has no ESLint config, saving formats **nothing** — the LSP fallback is deliberately disabled so **ts_ls** never silently reformats. If a JS file “isn’t formatting,” that’s why, not a bug.

:ConformInfo in a buffer shows exactly which formatter will run there.

5. Project-scale changes (search & replace, refactoring)

You have three escalating tools — pick the right one:

1. **g r n (LSP rename)** — for renaming a *symbol*. Semantically aware, cross-file, catches only real usages. Always try this first for identifiers.
2. **Quickfix + :cfd** — grep in the sidebar, Ctrl-q to quickfix, then **:cfd** %s/old/new/ge | update. Precise, previewable, scriptable.
3. **Spectre** — visual project-wide find & replace with per-match approval:

Key	Action
Space S r	Toggle Spectre panel
Space S w	Spectre pre-filled with word under cursor
Space S s (visual)	Spectre with the selection

Inside Spectre: **t f** toggles **fixed-strings** (literal) mode — flip it on when your search has regex-special characters like **.** or **(**. Replace individual matches or all at once.

TODO/FIXME/NOTE comments are highlighted everywhere (todo-comments); **:TodoQuick-Fix** collects every one in the project into the quickfix list.

6. Git without leaving Neovim

Hunk-level work (gitsigns) — the underrated one

Change markers live in the sign column (+, ~, _). The **Space h** family operates on **hunks** — individual blocks of change:

Key	Action
l c / l C	Jump to next / previous change
Space h p	Preview the hunk's diff inline
Space h s	Stage the hunk (works on a visual selection too)
Space h r	Reset the hunk — discard just that change
Space h S	Stage the whole buffer
Space h R	Reset the whole buffer
Space h b	Blame the current line (full commit info)
Space h d	Diff this file against the index
Space t b	Toggle inline blame on every line

The pro workflow this enables: you've made five edits but only three belong in this commit. **l c** through the hunks, **Space h s** the keepers, **Space h r** the debris. You've built a clean commit without touching the terminal.

Repo-level work (lazygit + Snacks)

Key	Action
Space g g	Lazygit in a big floating window — stage, commit, push, branch, rebase
Space g l	Lazygit log view
Space g L	Lazygit log for the current file
Space g h b	Blame line (Snacks version)
Space g o b	Open the current file/selection on GitHub in your browser — visual-select lines first to link a range
Space g e	Git status in the sidebar picker

Space g o b on a visual selection is the fastest “look at this code” link generator you own — use it when sharing with teammates or filing issues.

7. Debugging (nvim-dap, wired for Go)

DAP is configured with delve for Go (**dap-go**), UI included. Mason auto-installs adapters.

Key	Action
Space b	Toggle breakpoint (press and <i>pause</i> — see gotcha below)
Space B	Conditional breakpoint (prompts for the condition)
F5	Start / continue
F1	Step into
F2	Step over
F3	Step out
F7	Toggle the DAP UI (also shows the last session’s result)

The UI opens automatically when a session starts and closes when it ends. For Go, try it on a failing test: breakpoint on the assertion, F5, inspect locals in the UI instead of sprinkling `fmt.Println`.

8. Terminal, buffers, windows

Terminals

Key	Action
Ctrl-/ or Space r t	Toggle the Snacks terminal (bottom 40%, persists between toggles)
Space T	A plain 10-line split terminal
Esc Esc	Get out of terminal-insert mode back to normal mode

The Snacks terminal *toggles* — the session survives hiding it. Run your dev server or test watcher there and flip it in and out with `Ctrl-/,`.

Buffers (barbar — your “tabs”)

Key	Action
<code>Ctrl-, /</code>	Previous / next buffer
<code>Ctrl-.</code>	
<code>Ctrl-p</code>	Buffer pick — letters appear on each tab, type one to jump
<code>Ctrl-c</code> or <code>Space x</code>	Close buffer
<code>Space b o</code>	Close all but current
<code>Space b P</code>	Pin buffer (<code>Space b 0</code> closes all but pinned)
<code>Space Space</code>	Fuzzy-find open buffers

`Ctrl-p` buffer-pick is faster than cycling once you have 5+ buffers open. Pin the 2–3 files you keep returning to, then `Space b 0` to nuke the noise.

Windows

Key	Action
<code>Ctrl-h/j/k/l</code>	Move between splits
<code>:vsp / :sp</code>	Vertical / horizontal split
<code>Ctrl-Enter</code> (in Telescope)	Open result in a vertical split

9. Files on disk (Oil) — edit your filesystem like text

– opens the parent directory **as an editable buffer**. This is the part of your setup most worth mastering because it’s so unlike other editors:

- **Rename** a file: edit its line, `:w`.
- **Create** files/dirs: open a new line, type `name.ts` or `newdir/`, `:w`.

- **Delete:** `dd` the line, `:w` (goes to **trash**, not permanent — your config).
- **Move:** `dd` in one directory, `p` in another, `:w`.
- Batch all of the above in one write. Simple edits skip the confirmation prompt.

Every Oil buffer shows a two-line cheat header (your autocmd), but the keys to know: `Enter` open, `Ctrl-s/Ctrl-h` open in `v/h` split, `Ctrl-t` tab, `Ctrl-p` preview, `g.` toggle hidden, `gs` sort, `gx` open in Finder/app, `` cd` there, `g?` full help. `Space -` opens Oil floating instead of replacing your window. `Space r p` / `Space r P` copy the path of the entry under the cursor.

Mass-renaming ten files? `-`, then use *vim editing* — visual block, `:s`, macros — on the filenames, then `:w`. This is where Oil beats any file tree.

10. AI in-editor (Avante, Claude-powered)

Avante runs Claude (Sonnet) inside Neovim under the `Space a` prefix (default Avante bindings):

Key	Action
<code>Space a a</code>	Ask — opens the sidebar with your question
<code>Space a e</code>	Edit the visual selection via a prompt
<code>Space a t</code>	Toggle the Avante sidebar
<code>Space a r</code>	Refresh the sidebar
<code>Space a f</code>	Switch focus to/from the sidebar

It reads an `avante.md` file in your project root for per-project instructions if present. For quick “explain this / tweak this selection” moments, `Space a e` on a visual selection is the high-value one.

11. Gotchas specific to *your* config

These are the things `:h` won’t tell you because you built them:

- **Space b is two things.** Alone (press, then pause ~300ms) it toggles a DAP **breakpoint**. Followed quickly by another key it's the **buffer** prefix (Space b c close, Space b o only, ...). If breakpoints appear “randomly,” you hesitated mid-chord.
 - **Ctrl-p is context-dependent.** Globally it's barbar's buffer pick; inside an Oil buffer it's file preview (buffer-local wins).
 - **Visual p never yanks.** You remapped it — pasting over a selection keeps your register. If you *want* to swap, that's gone; yank explicitly first.
 - **JS/TS format-on-save silently does nothing without an ESLint config.** By design (see §4).
 - **The sidebar is exclusive.** Opening any picker closes whatever else was in the sidebar — one panel, always. q closes it.
 - **Telescope grep is literal, Snacks grep is regex (§1).**
 - **d still yanks** — you considered blackhole-delete and left it commented out in `keybindings.lua`. If deleted text keeps eating your register, either un-comment that block or get in the habit of yanking to a named register ("**ay**") before destructive edits.
 - **neo-tree is still installed** but unbound — `:Neotree` if you ever miss it.
-

12. Getting better: a four-week plan

Each week, pick the listed habits and *force* them — undo the old way when you catch yourself. One week per layer is enough for muscle memory to start.

Week 1 — Navigate by meaning, not by scrolling. Ban the mouse and the file tree except for orientation. Every file jump goes through Space F / Space G / Space Space. Every symbol jump goes through g r d → Ctrl-t, g 0 within files. Use relative line numbers (they're on): 7 j instead of mashing j. Toggle with Space t l if you want to check absolute numbers.

Week 2 — Operator + text object. No more v i w c when c i w works. Drill: c i ", c i a (argument), y a f (function call), d a), then the mini.ai specials c i n q (next quote) and surround (s a i w " / s d " / s r " '). Watch for the dot: after any of these, . repeats the change at the next spot — n . n . with a search is a mini-refactor.

Week 3 — Work at project scale. Every rename goes through g r n. One real refactor

via `grep` → `Ctrl-q` → `:cfd %s//.../ge | update`. One messy multi-file replace through Spectre. Fix diagnostics via `g r e` + `Space c a` instead of retyping by hand. Learn to read `Space s d` as your post-refactor checklist.

Week 4 — Git and debug in-editor. Build every commit with hunk staging (`l c`, `Space h s/h r`) or lazygit (`Space g g`) — terminal `git add .` is banned. Debug one real Go bug with breakpoints and the DAP UI instead of print statements. Blame before you edit unfamiliar code (`Space h b`).

Daily micro-habits (all four weeks):

- Forgot a binding? `Space` + wait (which-key), or `Space s k` — never leave the editor to look it up.
- Opened a big unfamiliar file? `z M` first, skim the skeleton, `z p` to peek.
- About to share code? Visual-select, `Space g o b`, paste the GitHub link.
- Something feels slow or repetitive three times in a row? That's a signal there's a binding for it — search `Space s k`.

Appendix: one-page cheat sheet

Find `Space F` files · `Space G` grep · `Space Space` buffers · `Space /` in-buffer · `Space e` tree · `Space s w` word · `Space s .` recent · `Space s r` resume · `Ctrl-q` → quickfix

Code `nav` `grd` def · `grr` refs · `gri` impl · `grt` type · `g0` file symbols · `gW` workspace symbols · `K` hover · `Ctrl-t/Ctrl-o` back · `Ctrl-i` forward

Fix `gre/grE` next/prev diagnostic · `Space ca` code action · `grn` rename · `Space q` loclist · `Space sd` all diagnostics · `Space f` format

Edit `ci"` `daf` `cia` textobjects · `sa/sd/sr` surround · `Cmd-/` comment · `Cmd-S` save · `visual p` paste-keep · `.` repeat

Folds `zM` close all · `zR` open all · `za` toggle · `zp` peek

Git `l c/[c` hunks · `Space hs` stage · `Space hr` reset · `Space hp` preview · `Space hb` blame · `Space gg` lazygit · `Space gob` open on GitHub · `Space ge` git status

Debug Space b breakpoint · Space B conditional · F5 run · F2 over · F1 into · F3 out · F7 UI

Buffers/Windows Ctrl-,/Ctrl-. prev/next · Ctrl-p pick · Space x close · Space bP pin · Ctrl-hjkl splits

Terminal Ctrl-/ toggle · Esc Esc to normal mode

Files – Oil · edit + :w to rename/create/delete · g. hidden · g? help

Search/replace Space Sr Spectre · tf literal mode · :cfdo %s/a/b/ge | update

AI Space aa ask · Space ae edit selection · Space at toggle

Maintenance :Lazy plugins · :Mason LSP tools · :LspInfo · :ConformInfo · :checkhealth